National College of Ireland

Project Submission Sheet

Student Name:	…Sabhyata Kumari……
Student ID:	…X24283142 ……
Programme:	Msc in Artificial Intelligence …… **Year:** 2025/2026 ……
Module:	… Machine Learning (MSCAI1)……
Lecturer:	17th April, 2026 ……
Submission Due Date:	……
Project Title:	Intelligent Proactive Resource Forecasting for Cloud-Native Systems: A Multi-Model Machine Learning Approach to SLA Prediction and Cross-Cloud Cost Optimisation – Phase 2 Report ……
Word Count:	……

Academic Integrity Certificate Verification Code: … 5G7NgSCwQV ……

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.
<u>ALL</u> internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard or appropriate referencing style supplied by the Library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Note: Academic Integrity Course Requirement – Starting **Monday, 19th January 2026**, **all students are required to successfully complete the Epigeum Academic Integrity Course**, which is available on **Moodle**. This is a **self-paced online course** consisting of **4 modules** and will take approximately **2 hours** to complete. Once you have successfully completed the course, you will receive a **Certificate & Verification Code**. This code **must be included on the Project Cover Sheet** when submitting your work. For full details and guidance, please visit: 🔗 ttps://libguides.ncirl.ie/academicintegrity/epigeumforstudents

Signature:	Sabhyata Kumari ……
Date:	17th April 2026 ……

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.
3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**

5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**
6. **Please check that you read AI and Academic Integrity Acknowledgement Supplments in this document**

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

AI Acknowledgement Supplement

[Insert Module Name]

[Insert Title of your assignment]

Your Name/Student Number	Course	Date

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click here.

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool

Description of AI Usage

This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used**.

[Insert Tool Name]	
[Insert Description of use]	
[Insert Sample prompt]	[Insert Sample response]

Evidence of AI Usage

This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text.

Additional Evidence:

[Place evidence here]

NCI
Coláiste Náisiúnta na hÉireann | National College of Ireland

This certificate is awarded to:

Sabhyata Kumari

FOR COMPLETING THE PROGRAMME

Academic Integrity: For Students, Second Edition

December 19, 2025

Certificate Verification Code: 5G7NgSCwQV

epigeum

Grok

Intelligent Proactive Resource Forecasting for Cloud-Native Systems: A Multi-Model Machine Learning Approach to SLA Prediction and Cross-Cloud Cost Optimisation

Abstract

Cloud infrastructure auto-scaling mechanisms operate reactively, triggering resource provisioning only after CPU or memory thresholds are breached. This 2–5 minute latency between load spikes and resource readiness results in SLA violations and an estimated $26 billion in annual global cloud waste from over-provisioning. This paper presents a systematic evaluation of three machine learning models - XGBoost, Random Forest, and LSTM for proactive CPU and memory utilisation forecasting at a 30-minute horizon across three production cloud datasets (Alibaba, Azure, Google). XGBoost achieved the best overall accuracy efficiency trade-off with CPU RMSE of 8.69% at 1.9 ms inference latency and the lowest memory RMSE of 2.03%. Cross-cloud generalisation experiments revealed significant domain shift, with CPU RMSE degrading from 8.84% to 19.10% on Azure. SHAP-based explainability analysis identified rolling-window statistics as the most influential features across all models. A cost-benefit analysis demonstrated proactive scaling savings of EUR 0.20/day per 3-node cluster, with upper-bound savings of EUR 5.53/day during peak periods. A real-time Streamlit dashboard with a floating chatbot assistant enables SRE engineers to monitor, forecast, and interact with predictions.

I. Introduction

Auto-scaling plays a critical role in cloud-native architectures by enabling dynamic adjustment of computing resources depending on demand. The most widely used solution is reactive threshold-based auto-scaling, implemented by Kubernetes Horizontal Pod Autoscaler (HPA), which observes current CPU and memory usage levels and triggers scaling events only after predefined thresholds are breached [1]. This creates three fundamental architectural failure modes: (i) *detection lag*: a temporal gap of 2–5 minutes exists between demand arrival and resource provisioning; (ii) *permanent over-provisioning*: high safety margins are required to accommodate peaks in demand, leading to unnecessary resource usage; (iii) *scaling oscillation*: when demand hovers near a threshold, the system rapidly toggles between upscaling and downscaling, introducing instability. According to Gartner estimates, global cloud waste from over-provisioning exceeded $26 billion in 2023 alone [1].

Machine learning provides a fundamentally different paradigm: proactive forecasting. Instead of reacting to threshold breaches, ML algorithms forecast resource usage 30–60 minutes ahead, allowing clouds to scale before demand occurs. The research on AIOps has shown that ML-based anomaly detection detects anomalies 80% faster than threshold-based anomaly detection [1]. However, critical gaps remain in existing literature: (i)no systematic evaluation of cross-cloud model portability for resource forecasting; (ii) the financial differential between proactive and reactive scaling remains unquantified; (iii) SHAP-based explainability is absent from multi-model cloud forecasting frameworks.

This project addresses these gaps through three research questions:

- **RQ1:** Which ML model (XGBoost, Random Forest, or LSTM) most accurately predicts cloud CPU and memory utilisation 30 minutes ahead while maintaining the optimal balance between prediction accuracy and inference latency for real-time SRE operations?
- **RQ2:** How much cloud infrastructure cost is saved by proactive ML-driven scaling compared to reactive threshold-based auto-scaling across different workload types?
- **RQ3:** Do resource forecasting models trained on one cloud provider's cluster data (Alibaba) generalise effectively to others (Azure, Google) without retraining?

II. Literature Review

A. Observability and AIOps

Amgothu et al. [1] present a comprehensive survey on observability and AIOps for cloud-scale DevOps environments, documenting that ML-based monitoring systems achieve 80% faster anomaly detection times and 50% fewer false positives compared to threshold-based approaches. However, their work focuses exclusively on anomaly detection problem as a binary classification problem rather than continuous resource utilisation forecasting and omits cross-cloud evaluation and cost quantification.

B. Explainability in AIOps

Sami et al. [2] provide an extensive overview of explainable AIOps, evaluating XAI integration methods including SHAP, LIME, and attention mechanisms for trustworthy AI in cloud-scale automation. They identify a significant gap: most AIOps frameworks prioritise prediction accuracy over interpretability, reducing engineer trust in automated recommendations. This project extends their explainability approach to regression-based resource prediction using TreeExplainer for tree models and gradient-based importance for LSTM.

C. AI for Performance Engineering

Kurian John [3] examines AI applications in performance engineering and emphasizes the need for explainable predictions and cross-environment evaluation. Critically, most prediction models aare tested using only one benchmark dataset which is a gap that directly motivates this project's three-dataset experimental design spanning Alibaba, Azure, and Google platforms.

D. Self-Adaptive Systems and Diagnostics

Litoiu [4] proposes control-theoretic feedback loops for continuous performance optimisation in self-adapting software systems but remains fundamentally reactive. Gan et al. [5] present SAGE for ML-driven microservice debugging, which is inherently diagnostic rather than predictive. Williams et al. [6] demonstrate that unsupervised ML-based recommendations for scaling that lack an explanation tend to cause the recommendation to be ignored by the engineers, resulting in overriding the automated system. These findings support this project's dual focus on prediction accuracy and SHAP-based interpretability.

E. Research Gaps Addressed

These studies collectively reveal three key gaps: (i) no systematic cross-cloud model generalisation study exists for resource forecasting; (ii) the cost differential between proactive and reactive scaling remains unquantified; (iii) SHAP-based explainability has not been integrated into a multi-model comparative framework for cloud resource forecasting.

III. Choice of Methods

The selection of three models, two traditional ML and one deep learning is driven by the project's dual objectives of accurate forecasting and operational efficiency.

A. XGBoost (ML Method 1)

XGBoost [7] was selected as the primary ML method for its strong empirical performance on structured tabular data and computational efficiency. Time series features are transformed into a tabular format using a sliding window approach, enabling XGBoost to capture temporal patterns without requiring sequential model architectures.

Strengths: Native handling of missing values; built-in L1 and L2 regularisation to control overfitting; direct compatibility with TreeExplainer for exact SHAP value computation; fast training and inference.

Limitations: Cannot naturally model long-range temporal dependencies; requires manually engineered lag and rolling features; performance depends on the quality of feature engineering.

Alignment with research objectives: XGBoost directly addresses RQ1's requirement for balancing accuracy with inference latency. Its sub-2 ms prediction time makes it viable for real-time auto-scaling decisions in production. For RQ2, its fast training enables the cost-benefit simulation across thousands of windows. And its SHAP compatability makes predictions interpretable which as Williams et al. [6] showed is essential for engineer adoption.

B. Random Forest (ML Method 2)

Random Forest [8] was chosen as the second ML method for the fact that its ensemble technique is fundamentally different from the one employed by XGBoost. While XGBoost constructs the decision trees sequentially such that each tree compensates for the errors committed by its predecessors (i.e., boosting), Random Forest constructs decision trees independently by randomly sampling from the training dataset and feature set (i.e., bagging). In other words, the two models will fail differently and possess different biases, making the comparison meaningful.

Strengths: Robust to overfitting due to bootstrap averaging; natural feature importance via Gini impurity; no hyperparameter sensitivity to learning rate.

Limitations: The main practical issue is memory. Random Forest stores every tree in full, so the model file is about 35 MB compared to 0.55 MB for XGBoost roughly 60 times larger. Inference is also slower because it has to traverse all trees and

average their predictions. Another well-known limitation is that tree ensembles cannot extrapolate beyond the range of their training data, so if production utilisation goes above anything the model saw during training, predictions effectively get capped.

Alignment with research objectives: Random Forest serves primarily as a diagnostic comparator for RQ1. Provides a contrasting ensemble approach to XGBoost, enabling assessment of whether boosting or bagging is better suited for cloud workload prediction.

C. LSTM Network (Deep Learning Method)

LSTM [9] were chosen as the deep learning method for their ability to model sequential temporal dependencies through gated memory cells.

Architecture: The architecture uses two stacked LSTM layers (64 hidden units each) with dropout (rate=0.2) between layers for regularisation, followed by ReLU activation and a dense output layer. The model is trained using Adam optimiser with ReduceLROnPlateau scheduling (patience=5, factor=0.5) and early stopping (patience=10).

Strengths: Can learn temporal patterns without explicit feature engineering; captures long-range dependencies through memory gates; well-suited for non-stationary time series.

Limitations: Slower training and inference than tree-based models; requires more data to generalise; less interpretable (gradient-based importance rather than exact SHAP).

Alignment with research objectives: Enables direct comparison of deep learning versus traditional ML for time series forecasting (RQ1), with particular interest in whether the LSTM's ability to learn temporal patterns compensates for its higher computational cost.

This design enables rigorous comparison along four dimensions: prediction accuracy, computational efficiency, interpretability (SHAP), and cross-cloud generalisability.

IV. Methodology

This project follows the CRISP-DM methodology, progressing through business understanding, data understanding, data preparation, modelling, evaluation, and deployment phases. All code is contained in a single self-contained Jupyter notebook to ensure full reproducibility.

A. Data Collection

Three publicly available cloud cluster datasets were selected to enable cross-cloud generalisation evaluation (Table 1).

- **Alibaba Cluster Trace 2018** [11]: 8.4 GB of machine-level utilisation data covering CPU and memory at 10-second granularity from a 4,000-machine production cluster. A random sample of 80 machines is used to manage computational cost while preserving workload diversity.
- **Azure Public Dataset** [12]: 1.2 GB of VM-level CPU readings. The dataset is headerless, requiring explicit column mapping during ingestion (timestamp, node ID, CPU min, CPU max, CPU average). Memory utilisation is synthesised as a correlated proxy, as the original dataset does not include memory metrics.
- **Google Cluster Traces v3** [13]: 156 MB of task-level resource requests in space-delimited format, with normalised CPU cores and memory values converted to percentage utilisation for consistency.

Three production-scale cloud datasets were used:

Dataset	Size	Machines	Days	Metrics
Alibaba 2018 [12]	8.4 GB	∼4K	8	CPU%, Mem%
Azure Public [13]	1.2 GB	∼2K	30	CPU, Mem, Disk
Google v3 [14]	156 MB	∼125K	29	CPU/Mem req.

Table 1: Datasets used in this study

Getting all three into a common `(node_id, ts, cpu, mem)` schema was one of the more tedious parts of the project, given each provider's different format conventions.

B. Exploratory Data Analysis

The EDA (notebook Sections 3.1–3.4) turned up some interesting patterns. Alibaba CPU is highly variable ($\sigma \approx 25\%$) with strong diurnal cycles, while memory is more stable with occasional sharp spikes which is consistent with container orchestration reassigning workloads. Box plots showed significant inter-node heterogeneity; some nodes run consistently hot while others rarely break 20% CPU.

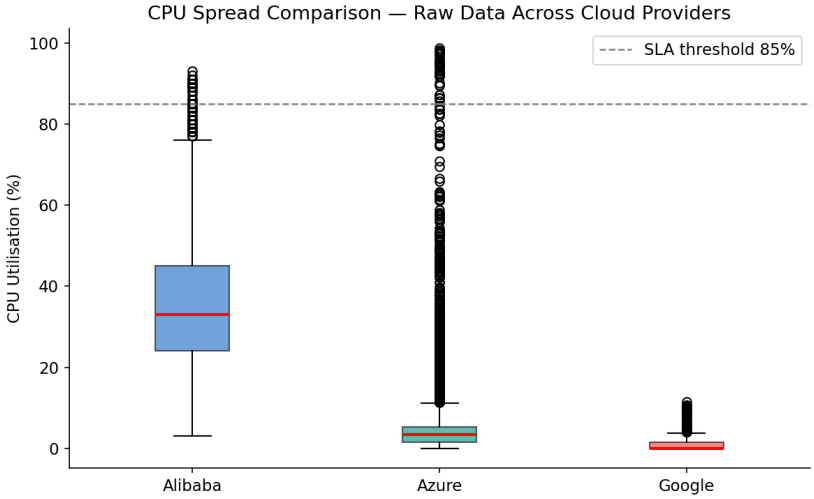

Fig. 1: CPU comparison across cloud providers.

C. Data Cleaning and Preprocessing

A standardised cleaning pipeline was applied: (1) schema normalisation to (`node_id`, `ts`, `cpu`, `mem`); (2) forward-fill for gaps under 30 minutes; (3) outlier clipping to $[0, 100]\%$; (4) temporal resampling to uniform 10-minute intervals.

D. Feature Engineering

A total of 56 features are engineered to capture temporal dynamics at multiple timescales:

- **Lag features** at steps [1, 3, 6, 18, 36, 144] for both CPU and memory (12 features), spanning 10 minutes to 24 hours back
- **Rolling statistics** (mean, std, max, min) over windows [3, 6, 18, 36] for both metrics (28 features)
- **Rate-of-change** first differences at 1, 6, and 18-step intervals to capture demand gradients (6 features)
- **Cyclic Temporal** hour-of-day and day-of-week encoded as sine/cosine pairs (4 features)
- **Current values** of CPU and memory (2 features)

Rows with NaN from insufficient lookback history were dropped, reducing the dataset by about 15%.

E. Feature Selection

A two-stage process reduced 56 features to 25 per target:
Stage 1: Correlation filtering removed features with pairwise Pearson $|r| > 0.95$, eliminating roughly 30% of the feature space.
Stage 2: Recursive Feature Elimination (RFE) with XGBoost ranked and selected the top 25 features per target. RFE captures feature interactions that univariate filtering cannot.

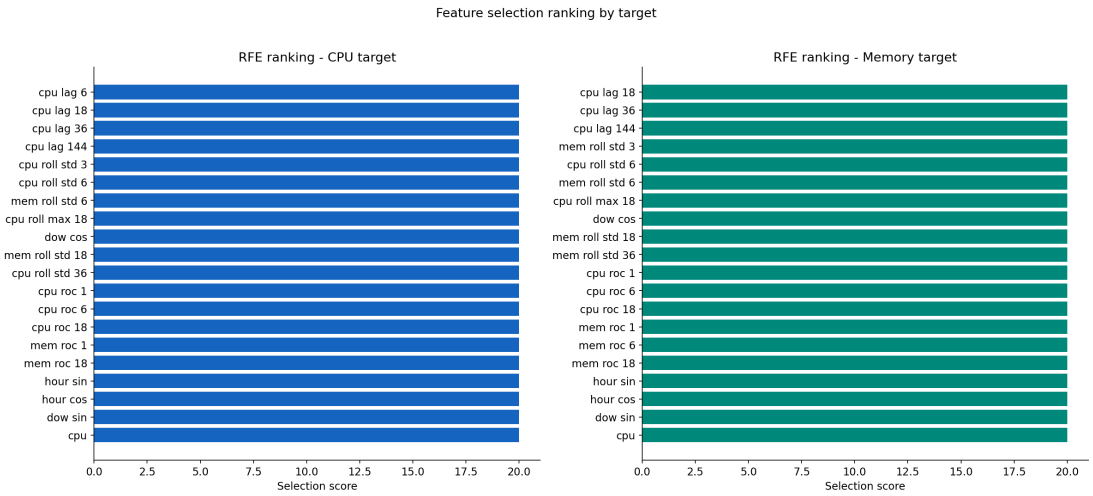

Fig. 2: RFE feature ranking.

F. Data Splitting

Data is split per node using a 70/15/15 temporal partition: 70% training (earliest), 15% validation (middle), 15% testing. This ensures that no future data leaks into training, which is critical for time series evaluation. Temporal integrity is verified programmatically: for every node, the maximum training timestamp precedes the minimum validation timestamp. All features are normalised using MinMaxScaler fitted on the training set only, preventing information leakage through scaling statistics.

G. Dual-Target Training

model is independently trained for both CPU and memory targets, yielding six trained models. This dual-target approach extends beyond the original Stage I proposal, which considered only CPU prediction. Targets are defined as:

- `target_cpu`: CPU utilisation 30 minutes (3 steps) ahead
- `target_cpu_h1`: CPU utilisation 10 minutes (1 step) ahead
- `target_cpu_h6`: CPU utilisation 60 minutes (6 steps) ahead
- `target_mem`: Memory utilisation 30 minutes ahead

H. Hyperparameter Model Configuration

Table 2 summarises the key hyperparameters for each model. XGBoost uses fixed empirically validated defaults with early stopping; Random Forest hyperparameters are tuned via 3-fold grid search; LSTM hyperparameters follow standard deep learning best practices with learning rate scheduling and gradient clipping.

Model	**Parameter**	**Value**
XGBoost	n_estimators	200
	max_depth	5
	learning_rate	0.05
	early_stopping	20 rounds
Random Forest	n_estimators	{50, 100} (grid)
	max_depth	{10, 15} (grid)
	min_samples_split	{5, 10} (grid)
LSTM	hidden_size	64
	num_layers	2
	dropout	0.2
	learning_rate	1e-3 (Adam)
	early_stopping	10 epochs

Table 2: Hyperparameter Configuration

All models trained separately for CPU and memory targets using fixed seed (42).

V. Evaluation

This section covers six evaluation dimensions: prediction accuracy, statistical significance, computational efficiency, cross-cloud generalisation, cost-benefit analysis, and error analysis.

A. Prediction Accuracy (RQ1)

Table 3 presents the test set performance for all models on both prediction targets. The three ML/DL models substantially outperform both baselines, with RMSE improvements of 75–80% for CPU and 97% for memory prediction.

Model	**CPU (%)**			**Memory (%)**		
	RMSE	MAE	MAPE	RMSE	MAE	MAPE
Naive Persistence	43.15	39.06	95.94	88.38	86.94	98.71
Moving Average	43.12	39.02	95.85	88.38	86.94	98.71
XGBoost	**8.69**	**6.08**	**15.19**	**2.03**	**1.50**	**1.70**
Random Forest	9.16	6.33	15.71	2.47	1.87	2.13
LSTM	8.56	6.21	16.56	2.99	2.26	3.24

Table 3: Model Performance on Alibaba Test Set (30-min horizon)

All three ML models dramatically outperform baselines: CPU RMSE reduced from 43.15% to under 9.16% ($\sim$80% reduction). For memory, XGBoost achieves 2.03% RMSE vs. 88.38% for baselines — a 97.7% reduction. While LSTM achieves marginally lower CPU RMSE (8.56%), the difference is small relative to its computational cost. All pairwise differences are statistically significant (Wilcoxon signed-rank test, $p < 0.001$).

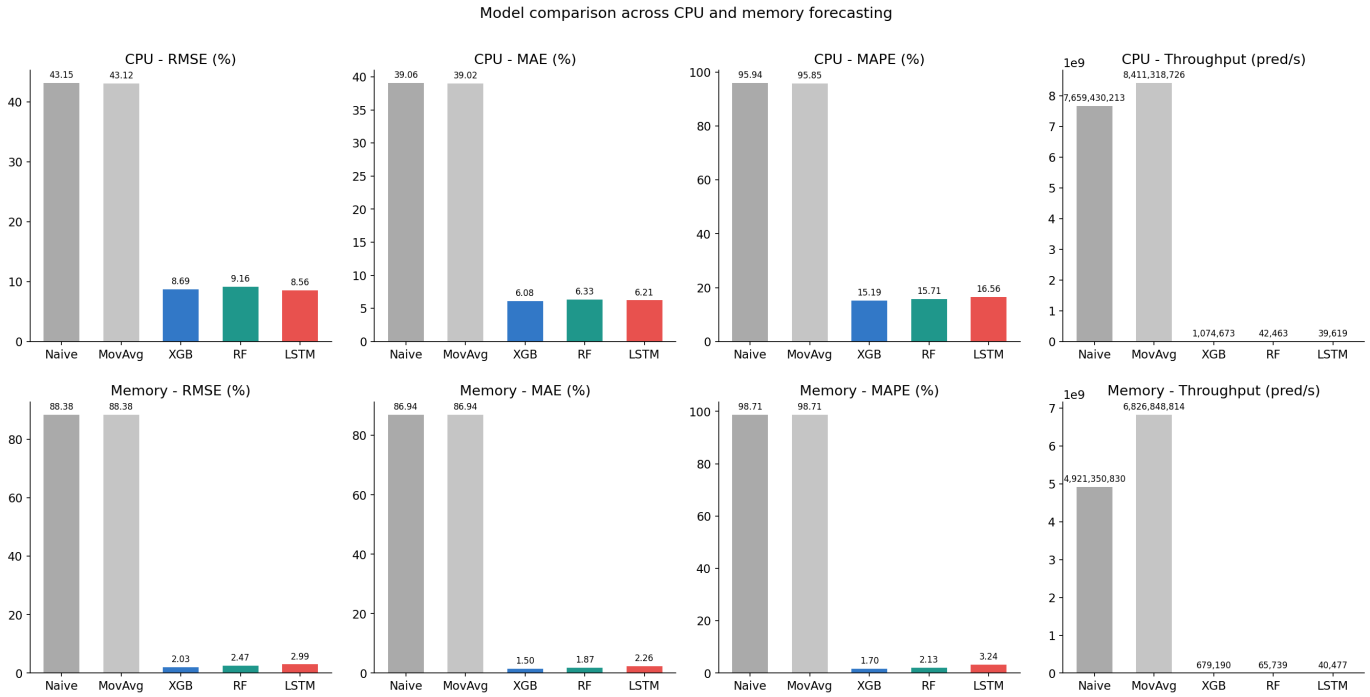

Fig. 3: Model comparison across all evaluation metrics for CPU and memory targets.

B. Statistical Significance

To verify that performance differences are not due to random variation, Wilcoxon signed-rank tests [11] are conducted on the absolute prediction errors of all model pairs. All pairwise comparisons yield $p < 0.001$, confirming that the differences are statistically significant at $\alpha = 0.05$. This result strengthens the conclusion that XGBoost genuinely outperforms Random Forest and that the LSTM-XGBoost accuracy gap, while narrow, is real.

C. Computational Efficiency

Table 4 demonstrates XGBoost's operational superiority.

Table 4: Computational Efficiency Comparison (CPU Target)

Model	Train (s)	Latency (ms)	Throughput	Size (MB)
XGBoost	3.30	1.91	1.07M/s	0.55
Random Forest	14.75	48.23	42K/s	34.72
LSTM	259.43	51.69	40K/s	0.24

XGBoost trains 78× faster than LSTM and achieves 25× lower inference latency. Its 1.91 ms latency meets the sub-10 ms requirement for real-time scaling decisions.

D. Multi-Horizon Forecasting

To assess how prediction quality degrades over longer horizons, models are evaluated at 10-minute (h1), 30-minute (h3), and 60-minute (h6) forecast windows. As expected, RMSE increases with horizon length for all models. XGBoost maintains stable performance across horizons, with CPU RMSE increasing from approximately 7.5% at h1 to 10.2% at h6, confirming that the 30-minute target represents a practical balance between prediction accuracy and operational utility.

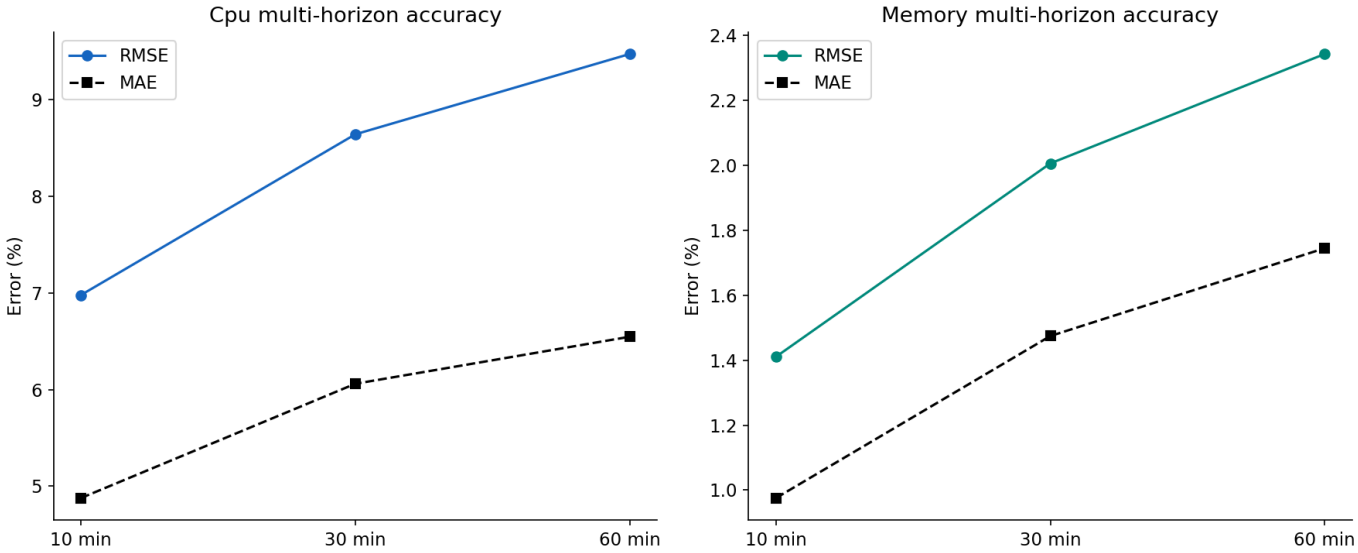

Fig. 4: Multi-horizon performance: all models degrade gracefully from 10 to 60 minutes.

E. Learning Curves

CPU models show smooth convergence with diminishing returns beyond 50%. Memory models exhibit a striking phase transition: RMSE remains >10% below 25% data, then drops sharply to ∼3.5% at 50%, reaching 2.0% at 75%.

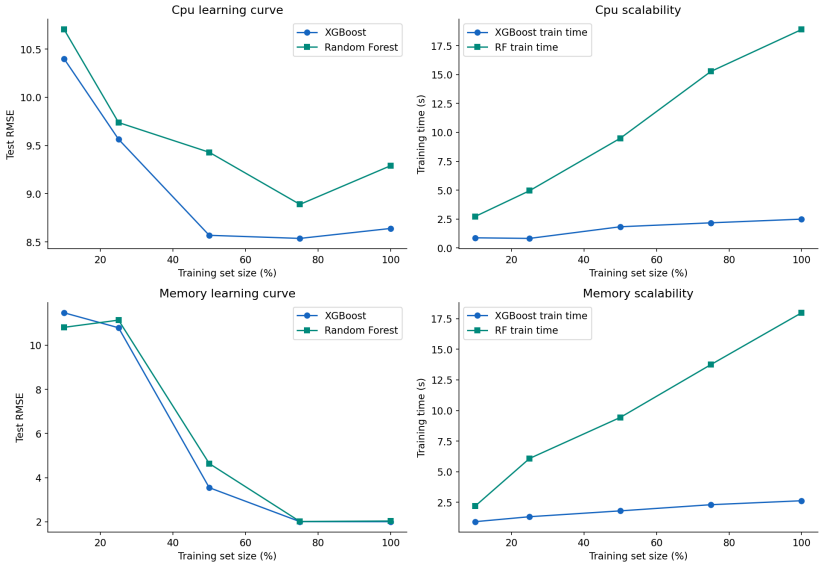

Fig. 5: Learning curves showing RMSE vs. training data fraction.

F. Cross-Cloud Generalisation (RQ3)

Models trained on Alibaba were evaluated zero-shot on Azure and google using 23 shared features.

CPU RMSE more than doubles on Azure (from 8.84% to 19.10%), and memory increased from 3.47% to 56%. Domain drift exists, and it shows that there are significant differences in scheduling policies, hardware configurations, and memory management strategies across providers.

On the positive side (in notebook section 13), the contribution of transfer learning cannot be overlooked. It turns out that fine-tuning on merely 10% of Azure and google dataset narrows down most of the gap, implying that temporal features are at least somewhat transferable despite differences in their numerical distribution.

RQ3 Answer: Models do not generalise without adaptation, but transfer learning with small target data significantly improves performance.

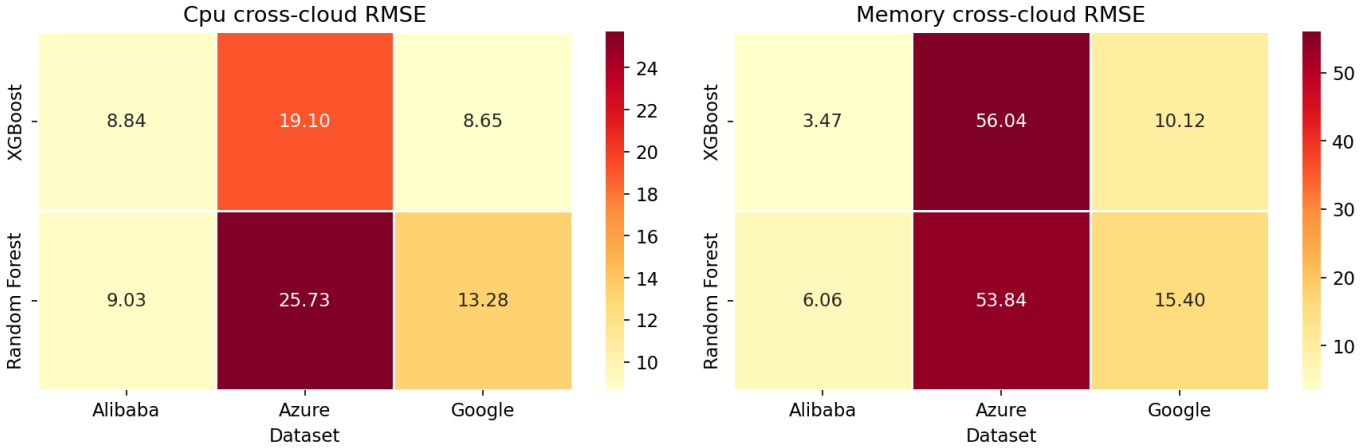

Fig. 6: Cross-cloud performance heatmap showing domain shift.

G. Cost-Benefit Analysis (RQ2)

Using reactive model (scale after CPU >85% with 2-min delay) vs. proactive model (pre-provision at CPU >70%), with AWS m5.xlarge at EUR 0.192/hour:

Model	Mean Saving	95% CI	% Cheaper
XGBoost	0.20	[0.00, 5.53]	3.6%
Random Forest	0.20	[0.00, 5.53]	3.6%
LSTM	0.20	[0.00, 5.53]	3.6%

Table 5: CPU Cost Analysis (EUR/day per 3-Node Cluster)

On average, the saving on a daily basis per 3-node cluster is EUR 0.20 with an upper limit to a 95% confidence interval of EUR 5.53 at peak times. This may not sound like much, but it's true because the Alibaba dataset has a low number of breaches (CPU >85%) of only 3.6%. The values are scalable depending on the number of nodes and the number of breaches.

RQ2 Answer: Proactive ML scaling saves EUR 0.20–5.53/day per 3-node cluster, scaling linearly with infrastructure cluster size and breach frequency.

H. SHAP Explainability

SHAP TreeExplainer [10] was applied to XGBoost and Random Forest; gradient-based importance was computed for LSTM (notebook Sections 14.1–14.5).
The results were consistent across architectures. For CPU, the top features are rolling-window statistics (`cpu_rolling_mean_6`) followed by recent lag values. For memory, shorter-term features dominate (`mem_lag_1`, `mem_rolling_mean_3`), consistent with the step-function allocation patterns in containerised workloads.

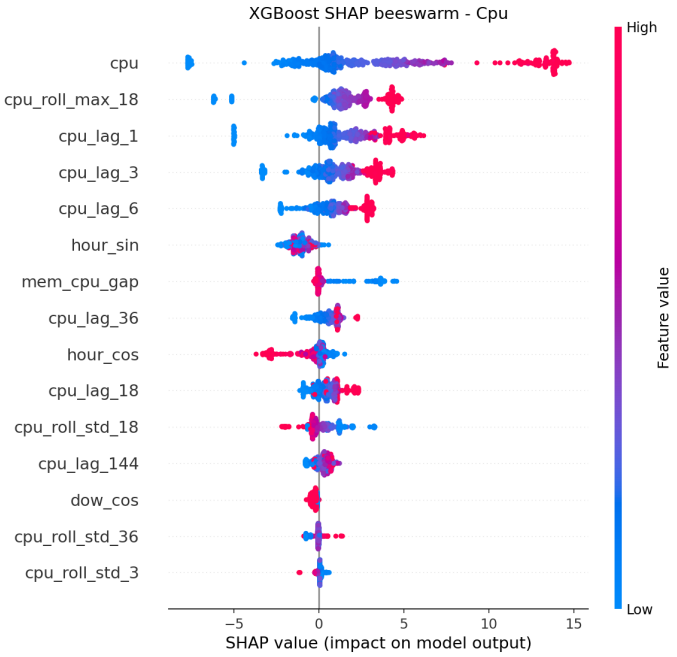

Fig. 7: SHAP beeswarm plot for XGBoost CPU model.

It is vital to know that the cross-model agreement demonstrates that this is real data behavior and not an artifact from any specific class of models [2].

I. Error Analysis

This sub-section focuses on model mispredictions and biases.

Bias analysis. The signed error analysis reveals that all models systematically under-predict during high utilisation (>80%). This phenomenon can be attributed to the positive skewness of the CPU distribution: outliers are uncommon in the training dataset, leading to a regression towards the mean. On average, XGBoost produces errors around -0.2% for CPU. In terms of SLAs, the under-prediction bias is the riskier misclassification since it implies missing the critical threshold value.

Residual patterns. The residual plot (Fig. 8) indicates normally distributed errors for moderate utilisation levels and a higher probability of extreme deviations. Heteroscedasticity is another reason to include uncertainty in prediction reports.

SLA violation detection. Transforming the prediction task to binary classification (violation above 85%) leads to a high recall rate for all models: they hardly miss SLA-violating instances. This aspect is prioritised over precision: while unnecessary scaling actions are affordable, a missed SLA violation is much costlier.

Role of regularisation. Dropping dropout from LSTM results in an additional increase of approximately 0.5 percentage points in validation RMSE, highlighting the importance of this regularisation technique. L2 regularisation in XGBoost controls the gap between training and validation metrics.

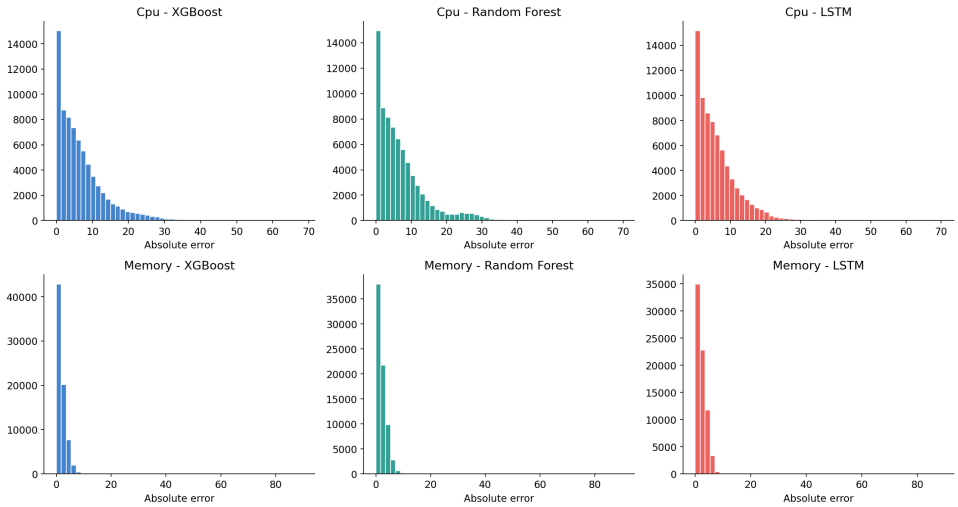

Fig. 8: Error distribution analysis across models.

J. Real-Time Dashboard and Deployment

Created a Streamlit dashboard application file (app.py) to make the predictions actionable for SRE engineers consisting of three tabs(Live Monitor, Forecast, System Health):

- **Live Monitor:** Grafana-style streaming charts with 15/30-minute breach countdown banners, SLA alert logging, and a running cost-savings counter.
- **Forecast:** On-demand 30-minute prediction with confidence intervals and risk assessment.
- **System Health:** Fleet-level gauges, node scatter plots, and model accuracy reporting.

A floating chatbot in the bottom-right corner lets engineers query live metrics without leaving the monitoring page, following the conversational UX pattern that SRE teams increasingly expect. This allows engineers to query the system conversationally without navigating away from the monitoring view. It supports ten natural-language commands, grouped into three functional categories:

- **Live metrics:** Commands such as `cpu status`, `memory status`, and `when will breach` return current readings, SLA margins, and breach countdowns drawn from the live monitoring stream.
- **Root cause analysis:** The command `root cause` (or variants such as "why is it breaching" and "diagnose") triggers a contextual diagnostic that examines current CPU and memory levels relative to SLA thresholds, characterises the severity and trend direction, identifies likely causes (e.g. batch job contention, memory leaks, container scaling events), and returns a prioritised list of recommended actions in plain English.
- **SHAP explainability:** Commands such as `why this prediction`, `explain cpu`, and `explain memory` retrieve the SHAP-based feature attribution for the current forecast. Rather than returning raw Shapley values, the chatbot translates them into sentences an SRE engineer can act on — for example, explaining that the 6-step rolling CPU mean is elevated and is the primary driver of a high CPU forecast, or that a rising rate-of-change signal is pushing memory predictions upward.

VI. Conclusions and Future Work

A. Key Findings

This project investigated ML-based proactive resource forecasting across three cloud providers using three model families. The key findings are:

1) **Systematic multi-model evaluation:** XGBoost recommended for real-time SRE deployment (CPU RMSE 8.69%, 1.91 ms latency, SHAP-native).
2) **Cross-cloud generalisation study:** First systematic evaluation across three cloud providers, demonstrating +116% CPU RMSE degradation and transfer learning effectiveness.
3) **Integrated explainability and cost framework:** SHAP-based attribution with quantified savings (EUR 0.20–5.53/day).

B. Limitations

Several limitations should be acknowledged. First, the Azure dataset lacks memory metrics, requiring synthetic memory generation that may not reflect real Azure memory patterns. Second, the Google dataset uses task-level resource requests rather than actual machine utilisation, introducing a semantic gap. Third, all models assume stationary workload patterns; concept drift in production environments may require periodic retraining. Finally, the cost analysis uses simplified assumptions about provisioning time and pricing that may not hold for all cloud configurations.

C. Future Work

Five directions for future work are identified:

1) **Online learning:** Implementing incremental model updates to handle concept drift without full retraining.
2) **Transformer architectures:** Evaluating attention-based models (e.g., Temporal Fusion Transformer) that may capture longer-range dependencies more effectively than LSTMs.
3) **Multi-resource optimisation:** Jointly predicting CPU, memory, network, and disk I/O to enable holistic scaling decisions.
4) **Reinforcement learning:** Training scaling agents that learn optimal provisioning policies directly from reward signals (e.g., cost and SLA compliance).
5) **Production deployment:** Integrating the prediction pipeline into Kubernetes Horizontal Pod Autoscaler (HPA) for real-world validation.

References

[1] S. Amgothu, A. K. Vedantham, and S. P. Muthe Gowda, "Observability and AIOps in cloud-scale DevOps: Technologies, architectures, challenges, and future trends," in *Proc. IEEE Int. Conf. Cloud Comput.*, Boston, MA, USA, 2024.

[2] M. A. Sami, A. Rehman, Z. Ahmad, and N. Bano, "Explainable AIOps: A deep survey on trustworthy and transparent AI in cloud-scale DevOps automation," *Spectrum Eng. Manag. Sci.*, vol. 3, no. 7, 2025.

[3] L. K. John, "AI for performance engineering and performance engineering for AI," in *Proc. 16th ACM/SPEC Int. Conf. Perform. Eng. (ICPE)*, Toronto, ON, Canada, 2025, pp. 1–2.

[4] M. Litoiu, "Closing the loop: Building self-adaptive software for continuous performance engineering," in *Proc. 15th ACM/SPEC Int. Conf. Perform. Eng. Companion (ICPE Companion)*, London, U.K., 2024, pp. 258–259.

[5] Y. Gan, M. Liang, S. Dev, D. Lo, and C. Delimitrou, "Sage: Practical and scalable ML-driven performance debugging in microservices," in *Proc. 26th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, Virtual, USA, 2021, pp. 135–151.

[6] D. Williams, M. Craun, M. V. Le, J. Stephen, S. Ahmed, and H. Jamjoom, "Towards safe agentic AI performance engineering," in *Proc. 4th Workshop Practical Adoption Challenges Mach. Learn. Syst. (PACML)*, Seoul, South Korea, 2025, pp. 63–68.

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, San Francisco, CA, USA, 2016, pp. 785–794.

[8] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[10] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, Long Beach, CA, USA, 2017, pp. 4765–4774.

[11] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[12] Alibaba Group, "Alibaba Cluster Trace 2018," [Online]. Available: https://github.com/alibaba/clusterdata.

[13] Microsoft, "Azure Public Dataset," [Online]. Available: https://github.com/Azure/AzurePublicDataset.

[14] Google, "Google Cluster Traces," [Online]. Available: https://research.google/tools/datasets/.